

Implementation Report: AWS Elastic Load Balancer (ELB) and Auto Scaling

Submitted By

Adnan Akhtar

Computer Science Engineering Student

1. Introduction

Cloud computing enables organizations to deploy scalable and highly available applications. As user traffic increases, a single server may become overloaded, resulting in poor performance and downtime.

To address this challenge, Amazon Web Services (AWS) provides Elastic Load Balancing (ELB) and Auto Scaling services. ELB distributes incoming traffic across multiple EC2 instances, while Auto Scaling automatically adjusts the number of instances according to workload demand.

This project explores the architecture, configuration process, and benefits of implementing ELB and Auto Scaling in AWS.

2. Project Objectives

The main objectives of this project are:

- To understand the concept of Elastic Load Balancing.
 - To study the working of Auto Scaling Groups.
 - To improve application availability and fault tolerance.
 - To distribute incoming traffic efficiently.
 - To automatically scale resources based on CPU utilization.
 - To reduce downtime and optimize resource usage.
-

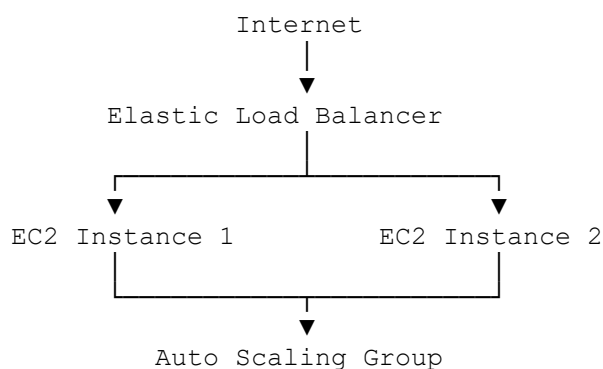
3. AWS Services Used

Service	Purpose
Amazon EC2	Virtual Server Hosting
Elastic Load Balancer (ELB)	Traffic Distribution

Service	Purpose
Auto Scaling Group (ASG)	Automatic Resource Scaling
Amazon CloudWatch	Monitoring and Metrics
Security Groups	Network Security
VPC	Virtual Private Cloud Networking

4. System Architecture

The system architecture consists of an Application Load Balancer placed in front of multiple EC2 instances.



The load balancer distributes requests among the available EC2 instances. When demand increases, Auto Scaling launches additional instances automatically.

5. Implementation Procedure

Step 1: Launch EC2 Instances

Two Amazon EC2 instances are launched to host a web application.

Configuration:

- Amazon Linux AMI
- t2.micro Instance Type
- HTTP (Port 80) Enabled
- SSH (Port 22) Enabled

A web server such as Apache is installed on both instances.

Step 2: Create a Target Group

A Target Group is created to register EC2 instances.

Configuration:

- Target Type: Instance
- Protocol: HTTP
- Port: 80

Both EC2 instances are added to the target group.

Step 3: Configure Elastic Load Balancer

An Application Load Balancer is created.

Configuration:

- Internet Facing
- IPv4
- HTTP Listener on Port 80
- Connected to Target Group

The load balancer routes incoming traffic to healthy EC2 instances.

Step 4: Create Launch Template

A Launch Template defines the configuration for future EC2 instances.

Includes:

- AMI Selection
- Instance Type
- Security Groups
- Startup Scripts

This template is used by Auto Scaling.

Step 5: Configure Auto Scaling Group

An Auto Scaling Group is created using the Launch Template.

Configuration:

- Minimum Capacity: 2
- Desired Capacity: 2
- Maximum Capacity: 4

The Auto Scaling Group is attached to the Load Balancer.

Step 6: Configure Scaling Policies

Target Tracking Scaling Policy is configured.

Metric:

- Average CPU Utilization

Threshold:

- 50% CPU Utilization

If CPU usage exceeds the threshold, additional EC2 instances are launched automatically.

When CPU utilization decreases, unused instances are terminated.

6. Working of the System

1. Users send requests to the application.
 2. Requests first reach the Load Balancer.
 3. ELB distributes traffic across available EC2 instances.
 4. CloudWatch continuously monitors CPU utilization.
 5. Auto Scaling automatically launches or terminates instances based on demand.
 6. The application remains available even during traffic spikes.
-

7. Benefits of ELB and Auto Scaling

High Availability

Traffic is distributed among multiple servers, reducing downtime.

Scalability

Resources automatically increase or decrease according to demand.

Fault Tolerance

If one server fails, requests are redirected to healthy servers.

Cost Optimization

Unused resources are terminated automatically, reducing infrastructure costs.

Improved Performance

Workloads are balanced across multiple servers.

8. Challenges Faced

- Understanding AWS networking concepts.
- Configuring security groups correctly.
- Linking Target Groups with Load Balancer.
- Setting Auto Scaling thresholds.
- AWS billing restrictions prevented live deployment.

To overcome this limitation, the implementation was studied and documented using AWS architecture and configuration guidelines.

9. Learning Outcomes

Through this project, the following concepts were learned:

- AWS Cloud Infrastructure
 - Elastic Load Balancing
 - Auto Scaling Groups
 - Launch Templates
 - CloudWatch Monitoring
 - High Availability Architecture
 - Cloud Resource Optimization
-

10. Future Enhancements

- HTTPS using SSL Certificates
- Route 53 Domain Integration
- Multi-Region Deployment

- CI/CD Pipeline using GitHub Actions
 - Docker Container Deployment
 - Kubernetes Integration
-

11. Conclusion

AWS Elastic Load Balancer and Auto Scaling are essential cloud services for building scalable, reliable, and highly available applications. ELB efficiently distributes incoming traffic across multiple EC2 instances, while Auto Scaling dynamically adjusts resources based on demand.

Although a live implementation could not be performed due to AWS billing restrictions, the architecture, configuration process, and operational workflow were thoroughly studied and documented. The project demonstrates a strong understanding of cloud scalability and modern infrastructure management practices.

Repository: AWS-ELB-AutoScaling-Implementation

Author: Adnan Akhtar